

MediaLedger 2.1.1: Developer and Rebuild Guide

Document date 2026-09-29. Release tag `v2.1.1`, commit `57c4e4f`.

Written for whoever rebuilds or extends MediaLedger without the original author at hand. Everything here was read from the repository or done on a real machine. Anything that could not be checked is marked UNVERIFIED and listed at the end.

Contents

1. Rebuilding from nothing

- 1.1 What you need
- 1.2 The commands
- 1.3 Clean build result

2. Repository map

3. Data model

- 3.1 Relationships
- 3.2 Migrations
- 3.3 Tables

4. Configuration

- 4.1 Files
- 4.2 Environment variables
- 4.3 Command line

5. Where things are stored at runtime

6. Tests

- 6.1 Test files that `npm test` does not run
- 6.2 Coverage

7. The release gate

8. Regenerating the screenshots and this package

9. Extension points

- 9.1 Add an operation
- 9.2 Add a page (a tab)
- 9.3 Add a Dashboard card
- 9.4 Add a database column or table
- 9.5 Teach the parser a new naming pattern
- 9.6 Support another video or subtitle format
- 9.7 Add an online source for episodes or posters
- 9.8 Add something to the family portal
- 9.9 Add a notification event

10. Open questions and unverified items

1. Rebuilding from nothing

1.1 What you need

Tool	Version	Why
Windows 10 or 11, 64-bit		The desktop installer is built for Windows x64. The server part also builds on Linux.
Node.js	22 (the release workflow); 24 also works	<code>node:sqlite</code> needs 22.5 or newer.
npm	Comes with Node	Installs from the lock file.
Git	Any recent	Source at the tag.
GNU tar	Comes with Git for Windows and with Windows 10 and newer	<code>tools/pack-server.js</code> calls <code>tar</code> .
Internet access		<code>npm ci</code> downloads the packages; <code>electron-builder</code> downloads Electron and NSIS on first use.

Not needed: a C or C++ compiler, Python, or any native build tool. The project has no native add-ons.

Build in a short folder, for example `C:\m1`. Windows limits a path to 260 characters, and the installer tool (NSIS) cannot open its own templates below a deeply nested folder. The symptom is `!include: could not open file` at the installer step, with everything before it passing. This happened while verifying this package and was resolved by moving the source to a shorter path.

1.2 The commands

```
git clone https://github.com/AxialForge/medialedger.git
cd medialedger
git checkout v2.1.1
npm ci
npm test
node tools/pack-server.js          # dist/medialedger-server-2.1.1.tar.gz and its checksum
npm run build:win -- --publish never # dist/medialedger-2.1.1-setup.exe
```

To run without building an installer:

```
npm start          # the desktop application
npm run web        # the web server on port 8080, data in %APPDATA%\MediaLedger-web
node src/server/server.js --data=<folder> --port=8090
node src/server/server.js --data=<folder> --set-password
```

1.3 Clean build result

Done on 2026-09-29T23:38:01Z by `docs/_tools/release-docs/clean-build.sh`: the source at `v2.1.1` was exported with `git archive` into an empty folder and built there, using nothing from the working copy.

Step	Command	Result	Seconds
install	<code>npm ci --no-audit --no-fund</code>	pass	8
test	<code>npm test</code>	pass	49
server-package	<code>node tools/pack-server.js</code>	pass	1
installer	<code>npm run build:win -- --publish never</code>	pass	49

Overall: PASS. 19 test suites reported "passed". Built with Node v24.18.0 and npm 11.16.0. Files produced: latest.yml, medialedger-2.1.1-setup.exe, medialedger-2.1.1-setup.exe.blockmap, medialedger-server-2.1.1.tar.gz, medialedger-server.tar.gz, medialedger-server.tar.gz.sha256.

The installer built this way is not byte-identical to the one on GitHub Releases: electron-builder embeds build times. It is functionally the same build of the same source.

2. Repository map

Path	Files	Purpose
src/main/	28	The core and the desktop shell. <code>service.js</code> registers every operation; the other files are one concern each.
src/preload.js	1	Builds the bridge the desktop pages call, from <code>bridge-shape.js</code> .
src/renderer/	18	The pages, shared by the desktop and the web server.
src/server/	5	The web server, its security, the family portal.
src/portal/	4	The family portal's own site. Never served by the main server.
server/	1	<code>install.sh</code> : the Raspberry Pi installer and updater.
tools/	4	<code>pack-server.js</code> (server package), <code>build-manual.js</code> (in-repository manual), <code>make-icon.js</code> , <code>movie-plan-xlsx.py</code> .
test/	23	Test suites and harnesses. Plain Node scripts with <code>assert</code> ; no test framework.
docs/	37	The in-repository manual, screenshots, the Raspberry Pi guide.
docs/_tools/release-docs/		The tools that made this package. See section 8.
docs/release-package/<version>/		This package.
build/	not tracked	Icons for the installer. The folder is in <code>.gitignore</code> , so a clone does not have it; see section 10.
.github/workflows/release.yml	1	The release workflow.
electron-builder.yml	1	Installer configuration.
CHANGELOG.md, README.md, CLAUDE.md, CONVENTIONS.md	4	History, overview, the maintainer notes with every hard-won gotcha, and house conventions.

Largest source files:

File	Lines
src/renderer/app.js	1991
src/main/service.js	865
src/main/db.js	545
src/renderer/styles.css	384
src/server/server.js	344
src/server/portal.js	321
src/main/scanner.js	303
src/main/plex.js	284

File	Lines
src/server/security.js	236
src/main/main.js	225
src/main/metadata.js	204
src/main/exportCsv.js	195

3. Data model

SQLite, one file, `medialedger.db`. Schema version **12** (the `user_version` of the database). 19 tables.

The base schema and the numbered migrations are in `src/main/db.js`. On start the database applies every migration above its own version, in order, inside a transaction each, **after copying itself to** `medialedger.db.backups/<stamp>-pre-migration-v<n>.db`.

3.1 Relationships

SQLite foreign keys are not declared; the relationships are by convention and kept by the code.

From	To	Meaning
<code>changes.scan_id</code>	<code>scans.id</code>	The scan that found the change.
<code>exports.scan_id</code>	<code>scans.id</code>	The scan the export followed.
<code>rename_items.batch_id</code>	<code>rename_batches.id</code>	The batch the file belongs to.
<code>rename_items.file_id</code>	<code>files.id</code>	The file that was renamed.
<code>plex_history.file_id</code>	<code>files.id</code>	The file that was played, when it is known.
<code>plex_history.account_id</code>	<code>plex_accounts.id</code>	Who played it. The name is also copied onto the row.
<code>files.plex_show_key</code>	<code>plex_shows.rating_key</code>	The Plex show of an episode.
<code>overrides (root_id, rel_path)</code>	<code>files (root_id, rel_path)</code>	The fix for a file.
<code>series_meta</code> , <code>series_prefs</code> , <code>user_ratings</code> , <code>title_tags</code> , <code>posters</code>	a title	Keyed by <code>library_type</code> plus the title key: the show name for series, <code>group_key</code> for movies.
<code>files.root_id</code>	<code>settings.json</code> roots	The root the file was found in.

3.2 Migrations

Version	Name
1	overrides, exports, ignored files, fix tracking
2	series metadata, duplicate keep marks, rename history
3	movie rename batches with undo journal, manual source override
4	adult flag, web videos, online + user ratings
5	plex links, ratings and watched state
6	media requests
7	genres, online tags and custom title tags

Version	Name
8	next airing episode
9	daily snapshots for trend cards
10	plex play history per account
11	collecting policy per series, plex sync detail
12	posters

3.3 Tables

changes

What each scan found: added, removed, modified, returned, renamed. Refers to `scans.id`.

Column	Type	Required	Default	Key
<code>id</code>	INTEGER			primary
<code>scan_id</code>	INTEGER	yes		
<code>ts</code>	TEXT	yes		
<code>kind</code>	TEXT	yes		
<code>library_type</code>	TEXT			
<code>path</code>	TEXT			
<code>detail</code>	TEXT			

Indexes: `idx_changes_scan` on `scan_id`.

exports

One row per CSV export.

Column	Type	Required	Default	Key
<code>id</code>	INTEGER			primary
<code>ts</code>	TEXT	yes		
<code>scan_id</code>	INTEGER			
<code>dir</code>	TEXT			
<code>files</code>	TEXT			
<code>rows</code>	INTEGER			
<code>trigger</code>	TEXT			

files

One row per file ever seen. The centre of the model: what the scan found, what ffpprobe measured, what the parser read, and what Plex says.

Column	Type	Required	Default	Key
id	INTEGER			primary
root_id	TEXT	yes		
library_type	TEXT	yes		
rel_path	TEXT	yes		
abs_path	TEXT	yes		
file_name	TEXT	yes		
ext	TEXT			
size	INTEGER			
mtime_ms	INTEGER			
first_seen	TEXT			
last_seen	TEXT			
missing	INTEGER		0	
parse_ok	INTEGER		0	
parse_note	TEXT			
show_name	TEXT			
season	INTEGER			
episode	INTEGER			
episode_end	INTEGER			
episode_title	TEXT			
movie_title	TEXT			
movie_year	INTEGER			
edition_tag	TEXT			
group_key	TEXT			
probed_at	TEXT			
probe_ok	INTEGER		0	
probe_error	TEXT			
container	TEXT			
duration_s	REAL			
bitrate_kbps	INTEGER			
width	INTEGER			
height	INTEGER			
resolution	TEXT			
fps	REAL			

Column	Type	Required	Default	Key
video_codec	TEXT			
video_profile	TEXT			
bit_depth	INTEGER			
hdr	TEXT			
audio_count	INTEGER			
audio_codecs	TEXT			
audio_langs	TEXT			
audio_channels	TEXT			
sub_count	INTEGER			
sub_codecs	TEXT			
sub_langs	TEXT			
sub_forced	INTEGER			
sidecar_subs	TEXT			
has_captions	INTEGER			
has_override	INTEGER		0	
ignored	INTEGER		0	
adult	INTEGER		0	
channel	TEXT			
video_id	TEXT			
upload_date	TEXT			
plex_rating_key	TEXT			
plex_title	TEXT			
plex_year	INTEGER			
plex_show_key	TEXT			
plex_guids	TEXT			
plex_user_rating	REAL			
plex_audience_rating	REAL			
plex_view_count	INTEGER			
plex_last_viewed	TEXT			
plex_view_offset_ms	INTEGER			
plex_section	TEXT			
plex_synced_at	TEXT			
plex_genres	TEXT			

Indexes: `idx_files_plex` on `plex_rating_key`; `idx_files_adult` on `adult`; `idx_files_group` on `group_key`; `idx_files_show` on `show_name`, `season`, `episode`; `idx_files_type` on `library_type`, `missing`; `sqlite_autoindex_files_1` (unique) on `root_id`, `rel_path`.

overrides

Manual fixes, keyed by root and relative path. Applied over the parser on every scan.

Column	Type	Required	Default	Key
<code>id</code>	INTEGER			primary
<code>root_id</code>	TEXT	yes		
<code>rel_path</code>	TEXT	yes		
<code>library_type</code>	TEXT			
<code>show_name</code>	TEXT			
<code>season</code>	INTEGER			
<code>episode</code>	INTEGER			
<code>episode_end</code>	INTEGER			
<code>episode_title</code>	TEXT			
<code>movie_title</code>	TEXT			
<code>movie_year</code>	INTEGER			
<code>edition_tag</code>	TEXT			
<code>ignore</code>	INTEGER		0	
<code>note</code>	TEXT			
<code>created</code>	TEXT			
<code>updated</code>	TEXT			
<code>keep</code>	INTEGER			
<code>source</code>	TEXT			

Indexes: `sqlite_autoindex_overrides_1` (unique) on `root_id`, `rel_path`.

plex_accounts

Plex account names by id.

Column	Type	Required	Default	Key
<code>id</code>	INTEGER			primary
<code>name</code>	TEXT			
<code>synced_at</code>	TEXT			

plex_history

One row per play, for every Plex account. `file_id` refers to `files.id`.

Column	Type	Required	Default	Key
history_key	TEXT			primary
rating_key	TEXT			
account_id	INTEGER			
account_name	TEXT			
device	TEXT			
type	TEXT			
title	TEXT			
show_title	TEXT			
season	INTEGER			
episode	INTEGER			
section	TEXT			
viewed_at	TEXT			
duration_s	REAL			
file_id	INTEGER			
library_type	TEXT			

Indexes: `idx_plex_history_key` on `rating_key`, `account_id`; `idx_plex_history_viewed` on `viewed_at`;
`sqlite_autoindex_plex_history_1` (unique) on `history_key`.

plex_shows

Show-level data from Plex.

Column	Type	Required	Default	Key
rating_key	TEXT			primary
section	TEXT			
title	TEXT			
year	INTEGER			
guids	TEXT			
user_rating	REAL			
audience_rating	REAL			
rating	REAL			
leaf_count	INTEGER			
viewed_leaf_count	INTEGER			

Column	Type	Required	Default	Key
content_rating	TEXT			
synced_at	TEXT			
genres	TEXT			

Indexes: `sqlite_autoindex_plex_shows_1` (unique) on `rating_key`.

plex_syncs

One row per Plex sync, with the per-library breakdown as JSON in `detail`.

Column	Type	Required	Default	Key
id	INTEGER			primary
ts	TEXT	yes		
sections	INTEGER			
items	INTEGER			
matched	INTEGER			
unmatched	INTEGER			
note	TEXT			
detail	TEXT			

posters

Which titles have a poster, its file and source.

Column	Type	Required	Default	Key
library_type	TEXT	yes		primary
title_key	TEXT	yes		primary
status	TEXT	yes		
source	TEXT			
file	TEXT			
mime	TEXT			
bytes	INTEGER			
fetches_at	TEXT			
tries	INTEGER		0	
note	TEXT			

Indexes: `sqlite_autoindex_posters_1` (unique) on `library_type`, `title_key`.

rename_batches

One row per rename batch.

Column	Type	Required	Default	Key
id	INTEGER			primary
ts	TEXT	yes		
mode	TEXT	yes		
layout	TEXT	yes		
status	TEXT	yes		
planned	INTEGER		0	
done	INTEGER		0	
failed	INTEGER		0	
undone	INTEGER		0	
finished	TEXT			
note	TEXT			

rename_items

The journal: one row per file in a batch. Refers to `rename_batches.id` and `files.id`.

Column	Type	Required	Default	Key
id	INTEGER			primary
batch_id	INTEGER	yes		
file_id	INTEGER			
root_id	TEXT			
from_rel	TEXT	yes		
to_rel	TEXT	yes		
from_abs	TEXT	yes		
to_abs	TEXT	yes		
size	INTEGER			
status	TEXT	yes		
error	TEXT			
ts	TEXT			
undone_ts	TEXT			

Indexes: `idx_rename_items_batch` on `batch_id`.

renames

Rename history from before batches existed.

Column	Type	Required	Default	Key
id	INTEGER			primary
ts	TEXT	yes		
root_id	TEXT			
from_rel	TEXT			
to_rel	TEXT			
ok	INTEGER			
error	TEXT			

requests

Media requests and their status.

Column	Type	Required	Default	Key
id	INTEGER			primary
created	TEXT	yes		
title	TEXT	yes		
kind	TEXT	yes		
year	INTEGER			
note	TEXT			
requested_by	TEXT			
status	TEXT	yes	'pending'	
admin_note	TEXT			
updated	TEXT			

scans

One row per scan with its counts and duration.

Column	Type	Required	Default	Key
id	INTEGER			primary
started	TEXT	yes		
finished	TEXT			
status	TEXT			
trigger	TEXT			
files_seen	INTEGER		0	

Column	Type	Required	Default	Key
added	INTEGER		0	
removed	INTEGER		0	
modified	INTEGER		0	
probed	INTEGER		0	
errors	INTEGER		0	
note	TEXT			
duration_ms	INTEGER			
threads	INTEGER			

series_meta

The online match of a series: source, expected episodes per season, status, rating, genres, next airing.

Column	Type	Required	Default	Key
id	INTEGER			primary
library_type	TEXT	yes		
show_name	TEXT	yes		
source	TEXT			
source_id	TEXT			
matched_title	TEXT			
status	TEXT			
seasons	TEXT			
total_episodes	INTEGER			
url	TEXT			
fetches_at	TEXT			
locked	INTEGER		0	
note	TEXT			
rating	REAL			
rating_votes	INTEGER			
genres	TEXT			
online_tags	TEXT			
next_airing	TEXT			
next_episode	TEXT			

Indexes: `sqlite_autoindex_series_meta_1` (unique) on `library_type`, `show_name`.

series_prefs

Collecting policy per series: muted, or from a season and episode onward.

Column	Type	Required	Default	Key
library_type	TEXT	yes		primary
show_name	TEXT	yes		primary
mute	INTEGER		0	
from_season	INTEGER			
from_episode	INTEGER			
note	TEXT			
updated_at	TEXT			

Indexes: `sqlite_autoindex_series_prefs_1` (unique) on library_type, show_name.

snapshots

One row per day describing the library, for the trend cards.

Column	Type	Required	Default	Key
day	TEXT			primary
ts	TEXT			
files	INTEGER			
bytes	INTEGER			
free_bytes	INTEGER			
series_tv	INTEGER			
series_anime	INTEGER			
movies	INTEGER			
missing_episodes	INTEGER			
watched_files	INTEGER			
linked_files	INTEGER			
pending_requests	INTEGER			
tagged	INTEGER			
captioned	INTEGER			

Indexes: `sqlite_autoindex_snapshots_1` (unique) on day.

title_tags

Your tags per title.

Column	Type	Required	Default	Key
id	INTEGER			primary
library_type	TEXT	yes		
title_key	TEXT	yes		
tag	TEXT	yes		
created	TEXT			

Indexes: `idx_title_tags` on `library_type`, `title_key`; `sqlite_autoindex_title_tags_1` (unique) on `library_type`, `title_key`, `tag`.

user_ratings

Your stars and note per title.

Column	Type	Required	Default	Key
id	INTEGER			primary
library_type	TEXT	yes		
title_key	TEXT	yes		
title	TEXT			
stars	REAL			
note	TEXT			
updated	TEXT			

Indexes: `sqlite_autoindex_user_ratings_1` (unique) on `library_type`, `title_key`.

4. Configuration

4.1 Files

File	Holds	Secrets inside
<code>settings.json</code>	Every option of the Settings page. Defaults are in <code>src/main/settings.js</code> ; the file holds only what differs.	The Plex token, the mail password, the GitHub token if one was entered.
<code>web.json</code>	Web server: accounts, password fingerprints, sessions, two-factor key, webhook and status keys, per-account preferences.	Yes. Written with mode 0600.
<code>portal.json</code>	Family portal: switch, port, addresses, invites and devices (fingerprints only).	Fingerprints only. Written with mode 0600.
<code>tls/cert.pem</code> , <code>tls/key.pem</code>	Web server: the self-signed certificate. Their presence is what turns HTTPS on.	The key.
<code>/etc/medialedger-cifs.cred</code>	Raspberry Pi: the credentials of the share.	Yes. Root only.
<code>/etc/medialedger-domain</code>	Raspberry Pi: the name given with <code>--domain</code> .	No.

4.2 Environment variables

Names only. None of them has a value in the repository.

Name	Purpose
<code>APPDATA</code>	Windows: where the desktop data folder lives.
<code>LOCALAPPDATA</code>	Windows: used to find an installed ffmpeg.
<code>MEDIALEDGER_DATA</code>	Data folder of the web server.
<code>MEDIALEDGER_HOST</code>	Address the web server binds to.
<code>MEDIALEDGER_PASSWORD</code>	Read once by <code>--set-password</code> so the password need not be typed. Secret: never commit a value.
<code>MEDIALEDGER_PORT</code>	Port of the web server.
<code>MEDIALEDGER_SHOT_PASSWORD</code>	Used by the screenshot pass to sign in to a web build. Secret.
<code>PATH</code>	Searched for ffprobe.
<code>NO_AUTO_UPDATE</code>	Set to 1 to disable the desktop update check.
<code>CSC_LINK</code>	Release workflow only: code-signing certificate. Secret. Not set.
<code>CSC_KEY_PASSWORD</code>	Release workflow only: its password. Secret. Not set.

4.3 Command line

Command	Switch	Purpose
Desktop	<code>--scan</code>	Scan, export and exit. What the Windows scheduled task runs.
Desktop	<code>--profile=<folder></code>	Use another data folder.
Desktop	<code>--screenshots=<folder></code>	Render every page and save a picture of each. The release gate.
Desktop	<code>--size=<width>x<height></code>	With <code>--screenshots</code> : the window size.
Desktop	<code>--url=<address></code>	With <code>--screenshots</code> : capture a web build instead.
Desktop	<code>--social=<file></code>	Render the social preview picture.
Web server	<code>--data=<folder></code>	Data folder.
Web server	<code>--port=<number></code>	Port. Default 8080.
Web server	<code>--host=<address></code>	Address to bind to. Default 0.0.0.0.
Web server	<code>--set-password</code>	Create or reset the account "admin".
Installer	<code>--port=</code> , <code>--share=</code> , <code>--domain=</code> , <code>--branch=</code> , <code>--https</code> , <code>--update-only</code> , <code>--auto-update</code> , <code>--no-auto-update</code>	See <code>server/install.sh</code> .

5. Where things are stored at runtime

What	Desktop (Windows)	Web server (Raspberry Pi)
Data folder	%APPDATA%\MediaLedger	/var/lib/medialedger
Program	Installed per user by the installer	/opt/medialedger
Database	medialedger.db (plus -wal, -shm)	same
Automatic database backups	medialedger.db.backups/	same
Backup sets	The folder chosen in Settings	same
Settings	settings.json	settings.json, web.json, portal.json
Log	medialedger.log	medialedger.log, security.log; also the systemd journal
CSV exports	exports/<stamp>/ and exports/latest/	same
Posters	posters/ or the folder chosen in Settings	same
Downloaded ffmpeg	tools/	not used; ffmpeg comes from the system
Restore in progress	restore-pending/, then pre-restore-<stamp>/	same
Certificate	not used	tls/, or tls-off/ when switched off
Caches	None on disk. The dashboard report is cached in memory for up to a minute.	same
In the browser	Theme, table or wall, filters	same, per device

6. Tests

```
npm test
```

Plain Node scripts that use `assert` and print "... passed". No framework, no configuration. `npm test` runs 15 files in sequence and stops at the first failure.

Suite	Covers
<code>test/parse.test.js</code>	see the file
<code>test/updater.test.js</code>	see the file
<code>test/features.test.js</code>	---- missing episodes ----
<code>test/rename.harness.js</code>	Exercises <code>applyRenames</code> against a temp folder + temp DB (never the real share).
<code>test/movieNamer.test.js</code>	Straight case: LiLTV → Web, probed 1080p, SDR, H264, English audio omitted
<code>test/movieRename.harness.js</code>	Exercises the movie rename executor against a temp folder + temp DB (never the real share).
<code>test/plex.test.js</code>	Pure Plex helpers: path mapping, mapping derivation, matching, flattening. No network.
<code>test/service.test.js</code>	Guards the shell/core split:
<code>test/paths.test.js</code>	<code>rel_path</code> keeps "/" separators in the database on every OS; <code>absOf</code> joins them for the local file system.
<code>test/security.test.js</code>	Web-shell security: TOTP against the RFC 6238 vectors, LAN detection, and the
<code>test/qr.test.js</code>	QR encoder checks. The full output was verified once against a reference decoder (OpenCV) for every
<code>test/tags.test.js</code>	notify: SMTP reply framing, and upgrade reasons
<code>test/restore.test.js</code>	Backup sets and restore: companions copied, sets pruned together, a bad backup refused, the swap on startup.
<code>test/portal.test.js</code>	The family portal from the outside: what gets in, what comes back, and what does not exist there.
<code>test/posters.test.js</code>	Posters against a stand-in Plex server: what is fetched, where it lands, what is skipped, and what is served.

6.1 Test files that `npm test` does not run

File	What it is	Status
<code>test/metadat a.fill.js</code>	Headless metadata fill for a profile DB (no Electron window). Usage:	A harness or a script that needs a real library or the network; run by hand.
<code>test/metadat a.live.js</code>	Live check against TVmaze and AniList (network). Not part of <code>npm test</code> .	A harness or a script that needs a real library or the network; run by hand.

File	What it is	Status
test/movieNamer.report.js	Dry report: run the movie naming engine over every movie row in a database	A harness or a script that needs a real library or the network; run by hand.
test/ratings.backfill.js	Backfill online ratings for series matched before ratings were stored. Network. Usage: node test/ratings.backfill.js <db>	A harness or a script that needs a real library or the network; run by hand.
test/rootcheck.test.js	Root reachability + folder browsing, without touching the network beyond localhost.	A real test suite that was never added to the script. Passes when run by hand (checked for this guide).
test/scan.harness.js	End-to-end: scan two small real roots into a temp DB, twice, then export CSVs.	A harness or a script that needs a real library or the network; run by hand.
test/webhook.test.js	Plex webhook: multipart parsing and applying events to linked files, on a throwaway database.	A real test suite that was never added to the script. Passes when run by hand (checked for this guide).
test/zip.test.js	The dependency-free zip writer must produce archives that the OS extractor accepts and that round-trip bytes.	A real test suite that was never added to the script. Passes when run by hand (checked for this guide).

6.2 Coverage

No coverage tool is configured, so there is no percentage to report. What can be said from reading the suites:

Area	Covered	Not covered
Parser and naming	Thoroughly: hundreds of real file names.	
Rename batches	End to end against a temporary folder: pre-flight, rename, journal, undo.	
Core and shell contract	Every operation must exist on both sides.	The behaviour of most operations.
Web security	Hashing, sessions, logout, roles, two-factor codes.	The HTTP layer itself.
Family portal	From the outside over HTTP: access, leaks, limits, the address check.	
Posters, backup and restore, tags, QR codes, Plex helpers	Yes, each with its own suite.	A real Plex server.
Scanner		Only through the harness, by hand.
The pages	The screenshot pass fails on any error a page logs.	No assertions about what a page shows.

7. The release gate

```
npm test
npx electron . --profile=<a data folder> --screenshots=<out folder> # exit code 3 = a page logged
an error
node tools/build-manual.js <out folder> docs/MediaLedger-Manual.docx
```

Then bump version in `package.json` and `package-lock.json`, add the section to `CHANGELOG.md`, commit, tag `vX.Y.Z` and push the tag. The workflow does the rest.

8. Regenerating the screenshots and this package

Everything is in `docs/_tools/release-docs/`. The screenshots show a generated, fictional library; no real data is involved.

File	Purpose
<code>make-demo-data.js</code>	Builds the fictional data folder. Fixed seed, so runs are repeatable.
<code>make-demo-posters.py</code>	Draws the demo posters.
<code>screens.js</code>	The list of screens: how to reach each and where it is defined.
<code>controls/*.js</code>	What is on each screen: every control with its description. The single source for callouts, tables and the inventory.
<code>uncaptured.js</code>	Short dialogs, the sidebar, keyboard shortcuts.
<code>capture.js</code>	Runs under Electron: rebuilds the demo data, starts the server, walks the screens, saves clean pictures and the position of every control. Masks the name and addresses of the computer.
<code>annotate.py</code>	Draws the numbered callouts.
<code>build-inventory.js</code>	Writes <code>ui_inventory.json</code> and checks every entry against the source.
<code>build-user-manual.js</code> , <code>build-release-overview.js</code> , <code>build-developer-guide.js</code>	Write the three documents.
<code>to-pdf.ps1</code> , <code>md-to-pdf.js</code>	Export to PDF.
<code>render-mermaid.js</code>	Renders the architecture diagram.
<code>clean-build.sh</code>	The clean rebuild check.
<code>make-archive.py</code>	Builds the reconstruction archive, its manifest and the secret scan, and checks every hash after unzipping.
<code>verify-archive.py</code>	Unzips the archive into a short folder and builds the source it contains.
<code>qa-check.py</code>	The final checks.
<code>merge-boxes.py</code> , <code>render-pages.py</code>	Helpers: merge a partial capture run; render PDF pages to a picture for checking by eye.

```
cd docs/_tools/release-docs && npm install          # once: the diagram renderer
cd ../../..
T=docs/_tools/release-docs; O=docs/release-package/<version>
npx electron $T/capture.js --data=$T/demo-data --out=$T/out
python $T/annotate.py $T/out $O/screenshots
node $T/build-inventory.js $O/ui_inventory.json $T/out/boxes.json
node $T/build-user-manual.js $O && node $T/build-release-overview.js $O
powershell -File $T/to-pdf.ps1 $O/USER_MANUAL.docx $O/RELEASE_OVERVIEW.docx
```

Needs Python 3 with Pillow and PyMuPDF, and Microsoft Word for the PDF export of the two Word documents.

To add a screen: add an entry to `screens.js`, describe its controls in `controls/`, run the capture with `--only=<id> --discover` and read `out/discover.json` for the selectors. A control the capture cannot find is reported, not skipped silently.

9. Extension points

9.1 Add an operation

1. In `src/main/service.js`: `h('area:name', (args) => ...)`.
2. Add it to `src/renderer/bridge-shape.js`. The pages then call `L.area.name(...)` in both shells.
3. If it only reads, add its name to the `READS` pattern in `service.js`, or it will needlessly drop the dashboard cache.
4. On the web server it is administrators-only until its name is added to `STANDARD` or `GUEST` in `src/server/server.js`. Add it to `SENSITIVE` if it changes files or security.
5. `npm test`: the contract test tells you what you forgot.

9.2 Add a page (a tab)

1. `views.name = async () => { view.innerHTML = ... }` in `src/renderer/app.js`.
2. A link in the sidebar in `src/renderer/index.html`: `<a href="#name" data-view="name">`.
3. Hide it from roles that may not see it, in `src/renderer/styles.css` (`body.role-guest [data-view="name"]`).
4. Add `['name', '#name']` to the list in `captureScreenshots` in `src/main/main.js`, so the release gate renders it.
5. Describe it in `tools/build-manual.js`.

9.3 Add a Dashboard card

Add one entry to `DASH_CARDS` in `src/renderer/app.js`: `{ type, group, label, help, sizes, def, guest, rule?, period?, list?, render(c) }`. `render` returns HTML. Data every card needs comes from `dashLoad()`; data only some cards need is fetched with `c.lazy(key, fn)` so it is requested once. Add the type to `DASH_DEFAULT` to show it to new accounts.

9.4 Add a database column or table

Append a migration to `MIGRATIONS` in `src/main/db.js` with the next version number. Never edit an existing migration. The backup before the upgrade is automatic. Mention the schema version in the changelog.

9.5 Teach the parser a new naming pattern

`src/main/parse.js`. Add the file names to `test/parse.test.js` first; the suite holds hundreds of real names and catches regressions.

9.6 Support another video or subtitle format

No code: the extensions are settings (`videoExtensions`, `subtitleExtensions`). To read a new property of a file, extend `src/main/ffprobe.js` and add the column with a migration.

9.7 Add an online source for episodes or posters

`src/main/metadata.js` for episodes: a lookup that returns `{ source, source_id, seasons, ... }`.

`src/main/posters.js` for posters: a function that returns an image, added to `fetchOne()`. Both pace themselves to about one request a second.

9.8 Add something to the family portal

`src/server/portal.js`. Add a route under `/p/`, and a `project...` function that copies **named fields only**. Never return a row from the core as it is. `test/portal.test.js` fails if a reply contains a path, an id or a Plex field.

9.9 Add a notification event

Add the name to `notify.events` in `src/main/settings.js` and to the list on the Settings page, and call `notifier.send('name', title, message)`.

10. Open questions and unverified items

Item	Status
build/ folder	<code>.gitignore</code> lists <code>build/</code> , and no file under it is tracked. <code>electron-builder.yml</code> names <code>build/icon.ico</code> . A clone therefore has no icon file ; electron-builder falls back to its default icon. The clean build confirms the installer still builds. To keep the icon, run <code>node tools/make-icon.js</code> or track the file.
Minimum hardware	Never measured.
Node versions	Built and tested on 22 (workflow) and v24.18.0 (this guide). Others untested.
Test coverage percentage	No tool configured.
Byte-identical rebuild	Not possible: the installer embeds build times.
Real Plex server	Not available to the documentation run; Plex code paths were exercised against a stand-in.